

Blocco 15 - Telemetria NoSQL con MongoDB

Architettura containerizzata con collector simulato e document database

NoSQL

30 min teoria + 90 min pratica

1. Dal ticketing alla telemetria

Riprendiamo il modello architetturale già visto: collector, DBMS, dati raw, dati curati e interrogazioni operative.

Obiettivo del blocco

Problema

Costruire un sistema riproducibile per raccogliere letture telemetriche da dispositivi energetici e ambientali.

Idea guida

La telemetria è un caso naturale per un database documentale: ogni messaggio è un documento con metadati stabili e metriche variabili.

Confronto con il sistema ticketing

Elemento	Ticketing	Telemetria
Collector	genera ticket	genera letture sensori
DBMS	PostgreSQL	MongoDB
Raw	eventi ticket grezzi	messaggi ricevuti dal sensore
Curated	ticket normalizzati	letture arricchite e validate
Dashboard	SQL su viste	aggregation pipeline

Perché MongoDB qui

- ▶ il documento conserva la forma del messaggio ricevuto;
- ▶ il campo `metrics` può cambiare per tipo dispositivo;
- ▶ gli indici supportano query per dispositivo, tempo e regione;
- ▶ l'aggregation pipeline permette KPI e trend;
- ▶ il laboratorio resta eseguibile in Docker.

Non è una scelta universale: è una scelta didattica coerente con documenti JSON e telemetria.

Servizio	Responsabilità
mongo	conserva database telemetry
telemetry-collector	genera una lettura ogni 10 secondi
mongo-express	permette ispezione web di database e collection
mongosh	esegue script di schema, collector e aggregazioni

```
docker compose -f docker-compose.telemetry.yml up -d  
docker compose -f docker-compose.telemetry.yml ps  
docker logs -f rdnosql-telemetry-collector
```

Servizio	Accesso
MongoDB	localhost:27018
Mongo shell	<code>docker exec -it rdnosql-telemetry-mongo mongosh</code>
mongo-express	<code>http://localhost:8081</code>
Credenziali web	<code>training/training</code>

Accesso a mongosh

```
docker exec -it rdnosql-telemetry-mongo mongosh
```

```
use telemetry
```

```
show collections
```

```
db.devices.countDocuments()
```

```
db.readings_curated.countDocuments()
```

2. Modello dati

Manteniamo la distinzione raw/curated già vista, ma cambiano forma del dato e linguaggio di interrogazione.

Collection principali

Collection	Contenuto
devices	anagrafica dispositivi e metadati stabili
readings_raw	messaggi ricevuti, quasi nella forma originale
readings_curated	letture arricchite con sito, regione e qualità
alerts	eventi generati da soglie
collector_runs	log del collector simulato

Perché raw e curated

- ▶ raw preserva il messaggio arrivato dal campo;
- ▶ curated rende interrogazioni e dashboard più semplici;
- ▶ gli errori di trasformazione possono essere tracciati;
- ▶ la pipeline è verificabile con conteggi raw-curated;
- ▶ la dashboard non dipende dal formato instabile del messaggio originale.

Collection devices

```
{  
  "device_id": "TR-roma-001",  
  "device_type": "transformer",  
  "site": "Roma Nord",  
  "region": "center",  
  "active": true,  
  "sampling_seconds": 60,  
  "tags": ["energy", "substation", "critical"]  
}
```

Collection readings_raw

```
{
  "raw_id": "seed-3",
  "received_at": ISODate("2026-05-13T08:10:02Z"),
  "source": "seed_loader",
  "payload": {
    "device_id": "TR-roma-001",
    "ts": ISODate("2026-05-13T08:10:00Z"),
    "metrics": {
      "temperature_c": 71.4,
      "load_kw": 910,
      "voltage_v": 19950
    },
    "status": "warning"
  }
}
```

Collection readings_curated

```
{
  "device_id": "TR-roma-001",
  "device_type": "transformer",
  "site": "Roma Nord",
  "region": "center",
  "ts": ISODate("2026-05-13T08:10:00Z"),
  "metrics": {
    "temperature_c": 71.4,
    "load_kw": 910,
    "voltage_v": 19950
  },
  "status": "warning",
  "quality": { "valid": true, "reason": "seeded" }
}
```

MongoDB permette validazione JSON Schema sulle collection.

- ▶ non è obbligatoria come in un DB relazionale;
- ▶ aiuta a bloccare documenti privi di campi minimi;
- ▶ non sostituisce la logica di qualità dati;
- ▶ va usata sui campi stabili, non su ogni possibile metrica variabile.

3. Script del laboratorio

Gli script sono pensati per essere copiati, rilanciati e discussi come nel sistema ticketing.

File	Uso
<code>nosql/telemetry_schema.js</code>	reset database, dati seed, indici
<code>nosql/telemetry_collector_tick.js</code>	inserisce una nuova lettura simulata
<code>nosql/telemetry_dashboard_queries.js</code>	aggiunge aggregazioni per KPI e dashboard

Reset e seed

```
docker exec rdnosql-telemetry-mongo \  
  mongosh /nosql/telemetry_schema.js
```

Lo script cancella e ricrea il database telemetry, inserisce dispositivi, letture iniziali, alert e indici.

Collector manuale

```
docker exec rdnosql-telemetry-mongo \  
  mongosh /nosql/telemetry_collector_tick.js
```

Il container telemetry-collector lo esegue in ciclo. Lanciarlo manualmente serve per mostrare che raw, curated e alert si aggiornano.

Controllo conteggi

```
docker exec rdnosql-telemetry-mongo mongosh --quiet --eval '
const t = db.getSiblingDB("telemetry");
printjson({
  devices: t.devices.countDocuments(),
  raw: t.readings_raw.countDocuments(),
  curated: t.readings_curated.countDocuments(),
  alerts: t.alerts.countDocuments()
});
'
```

4. Query operative

In MongoDB le query di dashboard diventano aggregation pipeline.

Ultima lettura per dispositivo

```
db.readings_curated.aggregate([
  { $sort: { device_id: 1, ts: -1 } },
  {
    $group: {
      _id: "$device_id",
      device_type: { $first: "$device_type" },
      site: { $first: "$site" },
      ts: { $first: "$ts" },
      status: { $first: "$status" },
      metrics: { $first: "$metrics" }
    }
  }
]);
```

KPI stato

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: "$status",
      readings: { $sum: 1 },
      devices: { $addToSet: "$device_id" }
    }
  },
  {
    $project: {
      readings: 1,
      device_count: { $size: "$devices" }
    }
  }
]);
```


Trend a bucket temporali

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: {
        bucket: { $dateTrunc: { date: "$ts", unit: "minute", binSize: 5 } }
      },
      readings: { $sum: 1 },
      warnings: { $sum: { $cond: [{ $eq: ["$status", "warning"] }, 1, 0] } }
    }
  },
  { $sort: { "_id.bucket": 1 } }
]);
```

- ▶ { device_id: 1, ts: -1 } per storia dispositivo;
- ▶ { device_type: 1, ts: -1 } per dashboard per famiglia;
- ▶ { region: 1, ts: -1 } per vista territoriale;
- ▶ { severity: 1, ts: -1 } per alert recenti.

Lettura degli indici

```
db.readings_curated.getIndexes();  
db.alerts.getIndexes();  
  
db.readings_curated.find(  
  { device_id: "TR-roma-001" }  
)  
.sort({ ts: -1 }).limit(5).explain("executionStats");
```

5. Attività pratica

Gli studenti devono avviare lo stack, osservare i dati, lanciare il collector e spiegare raw/curated.

- 1 avviare lo stack telemetry;
- 2 aprire mongo-express e trovare le collection;
- 3 eseguire `telemetry_schema.js`;
- 4 contare raw e curated;
- 5 eseguire due volte `telemetry_collector_tick.js`;
- 6 verificare che i conteggi crescano;
- 7 spiegare un documento `readings_curated`.

- ▶ L'architettura replica il modello ticketing ma usa MongoDB.
- ▶ Raw e curated restano separati anche in NoSQL.
- ▶ Il documento contiene metriche variabili e metadati utili alla query.
- ▶ Gli indici devono seguire dispositivo, tempo, regione e alert.